

AI 620 Project Report

Automated News Data Pipeline

Namra Nadeem (2528-0097)

Bimal Rehman (2528-0081)

Samia (2528-0104)

Sana Ahmed (2528-0001)

3rd May, 2026

1 Repository

<https://github.com/namranadeem-wq/DE-PROJECT-NEWS-PIPELINE>

2 Project Description

This project implements an end-to-end ELT pipeline for collecting, storing, processing, validating, analyzing, and serving news data. Data is collected from NewsAPI, GDELT, and WorldNewsAPI. The goal is to build a reproducible data engineering system that supports analytics, machine learning prediction, forecasting, and dashboard visualization.

3 Architecture Diagram

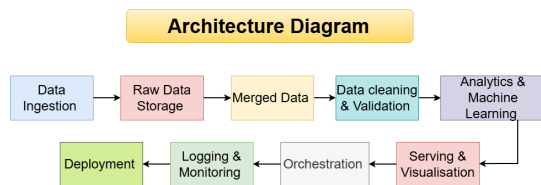


Figure 1: End-to-end architecture of the news data engineering pipeline.

4 Storage and Schema Design

The project uses both CSV files and SQLite. CSV files are useful for portability and manual inspection, while SQLite provides structured storage and querying. The cleaned dataset follows a structured schema designed for analytics and machine learning tasks as given Table 1

The url field acts as a unique identifier for deduplication.

5 Raw, Processed, and Cleaned Data

Raw data stores original API responses without modification, supporting reproducibility and backfills. Processed data contains merged and standardized records from all news sources.

Table 1: Dataset Schema

Column	Type	Description
title	string	News headline
description	string	Article summary
source_name	string	Source name
country	string	Country
published_at	datetime	Publish time
fetches_at	datetime	Ingestion time
url	string	Primary key
category	string	News category

Cleaned data is the final validated dataset used for ML and dashboard analytics. It removes duplicate URLs, fixes invalid timestamps, fills missing values, and keeps only usable rows.

6 Pipeline Explanation

Ingestion: Automated scripts fetch articles from APIs and handle authentication, pagination, request errors, and empty responses.

Processing: Different API schemas are converted into one common format and merged.

Validation: The pipeline checks missing values, duplicate URLs, invalid timestamps, required columns, and row counts.

Serving: The final dataset is shown using a Streamlit dashboard with filters, charts, ML results, and forecasting.

7 Error Handling and Logging

Error handling is implemented using try-except, API status code checks, timeout handling, and safe fallback outputs. Invalid dates are handled using `pd.to_datetime(errors="coerce")`. Critical missing fields such as title and URL are removed, while non-critical missing values are filled as Unknown.

Logs are stored in `logs/pipeline.log`. They record ingestion start/end, number of fetched articles, API failures, validation results, cleaning statistics, and final dataset shape. This improves debugging and monitoring.

8 Data Validation and Idempotency

Validation rules include:

- title and URL must not be null
- duplicate URLs are removed
- published at must be valid datetime
- required columns must exist
- missing author/source/category values are filled

The pipeline is idempotent because URL-based deduplication prevents repeated articles when the pipeline is re-run.

9 Orchestration, Scheduling, and Deployment

Prefect is used to orchestrate tasks such as ingestion, merging, cleaning, validation, and output generation. If a task fails, the flow stops instead of publishing bad data. Retry logic and scheduled hourly/daily runs can be configured in Prefect.

The project is Dockerized for reproducibility. Dependencies are listed in `requirements.txt`. The system can be run using:

- `docker build -t news-pipeline .`
- `docker run news-pipeline`
- `streamlit run serving/dashboard.py`

10 Analytics and Machine Learning

The dashboard provides analytical insights such as article counts, source distribution, category distribution, author frequency, country distribution, and daily article trends.

For machine learning, TF-IDF converts news text into numerical features, while N-grams capture important phrases such as “stock market” or “artificial intelligence”. The project includes category prediction and country prediction. Accuracy and confusion matrices are used for evaluation, while top TF-IDF terms provide feature-importance evidence.

11 ARIMA Forecasting

ARIMA forecasting is used to predict article volume over time by grouping articles by publication date. This demonstrates predictive analytics on time-series data. The model achieved about 82%, which is quite good considering that news headlines are very short. However, performance was hindered by short, noisy titles and heuristically inferred labels, which made classification difficult for minority classes with less data. However, forecasting is limited because the available dataset covers a short time window.

12 Why Great Expectations Was Not Used

Great Expectations was not used because the project scope was manageable with custom Python validation. The pipeline already performs null checks, duplicate checks, timestamp validation, schema checks, and row-count checks. In future work, Great Expectations can be added for formal validation suites and CI/CD gates.

13 Dashboard Screenshots

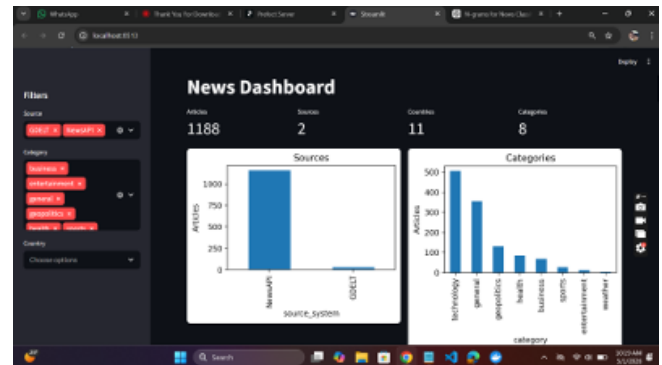


Figure 2: Dashboard overview

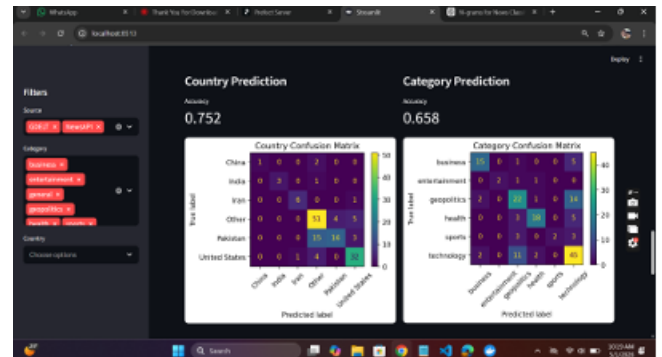


Figure 3: ML confusion matrices

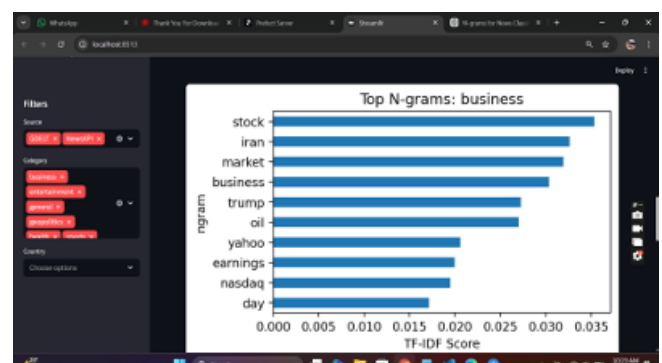


Figure 4: TF-IDF N-grams

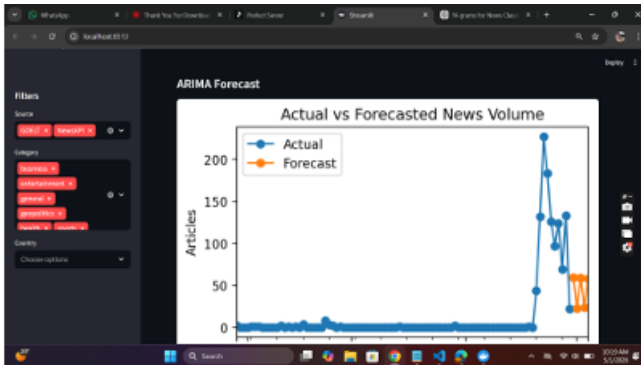


Figure 5: ARIMA forecast

14 Team Contributions

Namra Nadeem: News ingestion and dashboard.

Samia Documentation and Data cleaning

Bimal Validation logic and Docker implementation.

Sana Ahmed: Machine learning and analytics.

15 AI Usage Declaration

Tool: ChatGPT.

Used for: Debugging, explanation of concepts, report formatting, LaTeX assistance, and documentation refinement.

Extent: AI was used as an assistant; project implementation and final decisions were reviewed and adapted by the team.

16 Conclusion

This project demonstrates a complete data engineering workflow with automated ingestion, structured storage, cleaning, validation, orchestration, Docker deployment, logging, ML analytics, forecasting, and dashboard serving. Future work includes Great Expectations, stronger NLP models, cloud hosting, and real-time streaming.